# Municipal Complaint Management System

DBMS HACKATHON PROJECT SPECIFICATION & DATABASE DESIGN

**Database:** municipal_complaints.db
**RDBMS:** SQLite 3 (DB Browser Compatible)
**Architecture:** Python Flask • Relational SQLite

## 1. Problem Requirements

### 1.1 Background & Problem Statement

Municipal civic authorities encounter hundreds of citizen grievances daily relating to public amenities, solid waste accumulation, water pipeline leakages, broken asphalt, malfunctioning streetlights, and drainage overflow. Traditional complaint handling is severely plagued by fragmented paper logs, unrecorded status changes, delayed department allocation, lack of resolution accountability, and zero visibility for citizens.

The objective of this project is to architect and implement a simple, robust, relational database-backed **Municipal Complaint Management System**. The system provides an end-to-end mechanism to record complaints, allocate responsible engineering divisions, monitor lifecycle state transitions, verify formal engineering resolutions, and preserve an immutable chronological audit trail.

### 1.2 Functional Requirements

1. **Citizen Registration & Lookup:** Store citizen identifiers (Full Name, Phone Number) without requiring authentication, login passwords, or accounts. Returning citizens are automatically recognized by phone number.
2. **Standardized Grievance Classification:** Categorize public issues into discrete municipal divisions: *Garbage, Water Supply, Road, Street Light, and Drainage*.
3. **Geospatial Ward & Location Tracking:** Capture physical addresses, municipal ward/area designations, city names, and postal pincodes for precise field team dispatch.
4. **Departmental Workload & Assignment:** Maintain municipal engineering divisions with official nodal contact points (email, phone) and support dynamic complaint assignment.
5. **Complaint Lifecycle Progression:** Enforce structured status transitions: `SUBMITTED → ASSIGNED → IN_PROGRESS → RESOLVED / CLOSED / REJECTED`.
6. **Formal Resolution Certification:** When complaints reach `RESOLVED` status, mandate documentation of remedial action taken, certifying officer name, and resolution timestamp into a dedicated resolutions ledger.
7. **Comprehensive Audit Trail:** Automatically record every state transition into a historical log capturing old status, new status, administrative remarks, updating official, and exact timestamp.
8. **Real-time KPI Dashboard:** Present summary statistics (Total Complaints, Submitted, In Progress, Resolved) and department workload metrics to municipal desk officers.

### 1.3 Non-Functional & Scope Constraints

- **Zero Authentication Overhead:** Strictly excludes logins, sessions, passwords, roles, payment gateways, complex GIS/maps, or chat systems to keep focus purely on core DBMS principles.
- **Relational Integrity:** Strict enforcement of SQLite foreign keys ( `PRAGMA foreign_keys = ON;` ) and check constraints.
- **DB Browser Compatibility:** Native compatibility with **DB Browser for SQLite** for direct database inspection and SQL evaluations.

## 2. Entities

The database schema is structured into **7 distinct relational entities**, modeling both master data and transactional grievance lifecycles:

| ENTITY NAME | TABLE NAME | ROLE / CLASSIFICATION | DESCRIPTION & PURPOSE |
|---|---|---|---|
| **Citizen** | `citizens` | Master Entity | Represents the resident lodging the grievance; identified by mobile number and name. |
| **Category** | `categories` | Master Entity | Standard municipal domain classification (Garbage, Water Supply, Road, Street Light, Drainage). |
| **Location** | `locations` | Master Entity | Geographic physical site of grievance including street address, ward/area, city, and pincode. |
| **Department** | `departments` | Master Entity | Municipal administrative division responsible for remediating the complaint (e.g. Sanitation, Electrical). |
| **Complaint** | `complaints` | Core Transaction Entity | Central grievance record linking citizen, category, location, and department with priority and status. |
| **Complaint History** | `complaint_history` | Historical / Audit Entity | Immutable chronological log capturing all status transitions, remarks, officers, and timestamps. |
| **Resolution** | `resolutions` | Transaction / Certification | Detailed engineering rectification record, certifying officer, and completion timestamp for resolved issues. |

## 3. Attributes & Data Dictionary

The attribute specifications, data types, nullabilities, default values, and semantic roles for all 7 tables are detailed below:

### 3.1 Table: citizens

| ATTRIBUTE | DATA TYPE | CONSTRAINTS | DESCRIPTION |
|---|---|---|---|
| `citizen_id` | INTEGER | `PK` AUTOINCREMENT | Surrogate primary key uniquely identifying a citizen. |
| `citizen_name` | TEXT | NOT NULL | Full legal name of the reporting citizen. |
| `phone` | TEXT | NOT NULL | Citizen contact phone number used for complaint tracking and citizen reuse. |

### 3.2 Table: categories

| ATTRIBUTE | DATA TYPE | CONSTRAINTS | DESCRIPTION |
|---|---|---|---|
| `category_id` | INTEGER | `PK` AUTOINCREMENT | Surrogate primary key for the category. |
| `category_name` | TEXT | NOT NULL, `UNIQUE` | Category name (e.g. Garbage, Water Supply, Road, Street Light, Drainage). |
| `description` | TEXT | NULL | Summary scope of municipal grievances falling under this category. |

### 3.3 Table: locations

| ATTRIBUTE | DATA TYPE | CONSTRAINTS | DESCRIPTION |
|---|---|---|---|
| `location_id` | INTEGER | `PK` AUTOINCREMENT | Surrogate primary key for the coverage location. |
| `address` | TEXT | NOT NULL | Street name, crossroad, landmark, or house/plot number. |
| `area` | TEXT | NOT NULL | Municipal ward or neighborhood area name (e.g. Indiranagar, Jayanagar). |
| `city` | TEXT | NOT NULL | City jurisdiction (e.g. Bengaluru). |
| `pincode` | TEXT | NULL | 6-digit postal index number. |

### 3.4 Table: departments

| ATTRIBUTE | DATA TYPE | CONSTRAINTS | DESCRIPTION |
|---|---|---|---|
| `department_id` | INTEGER | `PK` AUTOINCREMENT | Surrogate primary key for the municipal department. |
| `department_name` | TEXT | NOT NULL, `UNIQUE` | Official title of the municipal engineering/administrative department. |
| `contact_email` | TEXT | NULL | Departmental official grievance email address. |
| `contact_phone` | TEXT | NULL | Nodal officer helpline phone number. |

### 3.5 Table: complaints

| ATTRIBUTE | DATA TYPE | CONSTRAINTS | DESCRIPTION |
|---|---|---|---|
| `complaint_id` | INTEGER | PK AUTOINCREMENT | Unique complaint registration number. |
| `citizen_id` | INTEGER | NOT NULL, FK | References `citizens(citizen_id)`. |
| `category_id` | INTEGER | NOT NULL, FK | References `categories(category_id)`. |
| `location_id` | INTEGER | NOT NULL, FK | References `locations(location_id)`. |
| `department_id` | INTEGER | NULL, FK | References `departments(department_id)`; assigned upon allocation. |
| `title` | TEXT | NOT NULL | Summary subject of the civic grievance. |
| `description` | TEXT | NOT NULL | Elaborated description of the problem, severity, and duration. |
| `priority` | TEXT | NOT NULL, CHECK, DEFAULT 'MEDIUM' | Must be in ('LOW', 'MEDIUM', 'HIGH', 'URGENT'). |
| `status` | TEXT | NOT NULL, CHECK, DEFAULT 'SUBMITTED' | Must be in ('SUBMITTED', 'ASSIGNED', 'IN_PROGRESS', 'RESOLVED', 'CLOSED', 'REJECTED'). |
| `submitted_at` | DATETIME | DEFAULT CURRENT_TIMESTAMP | Automatic timestamp when the complaint is lodged. |
| `resolved_at` | DATETIME | NULL | Timestamp populated when status is updated to 'RESOLVED'. |

### 3.6 Table: complaint_history

| ATTRIBUTE | DATA TYPE | CONSTRAINTS | DESCRIPTION |
|---|---|---|---|
| `history_id` | INTEGER | PK AUTOINCREMENT | Surrogate primary key for the audit record. |
| `complaint_id` | INTEGER | NOT NULL, FK | References `complaints(complaint_id)`. |
| `old_status` | TEXT | NULL | Previous status (NULL when initially lodged as SUBMITTED). |
| `new_status` | TEXT | NOT NULL | New status resulting from the administrative update. |
| `remarks` | TEXT | NULL | Official notes, reason for transition, or inspection observations. |
| `updated_by` | TEXT | NULL | Name or official role of the person recording the update. |
| `updated_at` | DATETIME | DEFAULT CURRENT_TIMESTAMP | Exact timestamp when the status modification occurred. |

### 3.7 Table: resolutions

| ATTRIBUTE | DATA TYPE | CONSTRAINTS | DESCRIPTION |
|---|---|---|---|
| `resolution_id` | INTEGER | PK AUTOINCREMENT | Surrogate primary key for the resolution certificate. |
| `complaint_id` | INTEGER | NOT NULL, UNIQUE, FK | References `complaints(complaint_id)`; enforces strict 1:1 cardinality. |
| `resolution_description` | TEXT | NOT NULL | Detailed description of engineering actions taken to remediate the complaint. |
| `resolved_by` | TEXT | NOT NULL | Name, title, or designation of the certifying field engineer/inspector. |
| `resolved_at` | DATETIME | DEFAULT CURRENT_TIMESTAMP | Official date and time when the resolution was certified. |

## 4. Keys & Relational Schema

The integrity of the relational model is enforced through primary keys, alternate/unique keys, foreign keys, and referential constraints:

### 4.1 Primary Keys (PK)

- `citizens.citizen_id`: Uniquely identifies citizen profiles.
- `categories.category_id`: Uniquely identifies grievance categories.
- `locations.location_id`: Uniquely identifies municipal physical locations.
- `departments.department_id`: Uniquely identifies municipal departments.
- `complaints.complaint_id`: Uniquely identifies complaint transactions.
- `complaint_history.history_id`: Uniquely identifies individual audit events.
- `resolutions.resolution_id`: Uniquely identifies formal resolution records.

### 4.2 Candidate & Unique Keys (AK)

- `categories.category_name` (UNIQUE): Prevents duplicate category definitions.
- `departments.department_name` (UNIQUE): Enforces distinct municipal department entities.
- `resolutions.complaint_id` (UNIQUE): Ensures exactly **one** formal resolution record per complaint.
- `citizens.phone`: Natural candidate key utilized by application logic to link repeated complaints by the same citizen.

### 4.3 Foreign Keys (FK) & Cardinalities

| FOREIGN KEY | REFERENCED TABLE & COLUMN | CARDINALITY | BUSINESS SEMANTICS |
|---|---|---|---|
| `complaints(citizen_id)` | `citizens(citizen_id)` | 1 : N | One citizen can register many complaints; every complaint must link to a valid citizen. |
| `complaints(category_id)` | `categories(category_id)` | 1 : N | Each complaint belongs to exactly one category; a category classifies multiple complaints. |
| `complaints(location_id)` | `locations(location_id)` | 1 : N | One physical location can accumulate multiple grievances over time. |
| `complaints(department_id)` | `departments(department_id)` | 1 : N (Optional) | Nullable initially; assigned when allocated to a department for field remediation. |
| `complaint_history(complaint_id)` | `complaints(complaint_id)` | 1 : N | One complaint produces an append-only chronological chain of audit records. |
| `resolutions(complaint_id)` | `complaints(complaint_id)` | 1 : 1 | A complaint has at most one formal engineering resolution certificate. |

## 5. Business Rules

The municipal system enforces explicit business logic across database constraints and backend API controllers:

### Rule 1: Domain Integrity via CHECK Constraints

- **Priority Domain Constraint:** Priority values are strictly restricted to 4 enumerated levels: `CHECK(priority IN ('LOW', 'MEDIUM', 'HIGH', 'URGENT'))`
- **Status Domain Constraint:** The lifecycle status must strictly belong to the defined states: `CHECK(status IN ('SUBMITTED', 'ASSIGNED', 'IN_PROGRESS', 'RESOLVED', 'CLOSED', 'REJECTED'))`

### Rule 2: Default Initialization & Timestamps

- Every newly lodged grievance automatically defaults to `status = 'SUBMITTED'`.
- If no priority is chosen by the citizen, it automatically defaults to `priority = 'MEDIUM'`.
- Creation timestamps ( `submitted_at` , `updated_at` , `resolved_at` ) are generated server-side using `CURRENT_TIMESTAMP`.

### Rule 3: Mandatory Citizen & Classification Data

- A complaint cannot be filed anonymously. Citizen Full Name, Phone Number, Category ID, Location ID, Title, and Description are non-nullable ( `NOT NULL` ).
- No login credentials (passwords, emails) are enforced, enabling frictionless public reporting.

### Rule 4: Department Assignment & Progression Lifecycle

- Grievances enter the system as `SUBMITTED` without mandatory departmental allocation.
- Progressing a grievance to `ASSIGNED` or `IN_PROGRESS` requires designating an active municipal department ( `department_id` ).

### Rule 5: Mandatory Resolution Certification Rule

> **Resolution Enforcement:** A complaint CANNOT be transitioned to `RESOLVED` status without simultaneously supplying a descriptive resolution summary ( `resolution_description` ) and certifying officer's name ( `resolved_by` ).

- When marked `RESOLVED` , the database updates `complaints.resolved_at = CURRENT_TIMESTAMP`.
- Concurrently, an entry is inserted into the `resolutions` table with foreign key linkage to the complaint.
- The `UNIQUE` constraint on `resolutions.complaint_id` guarantees that duplicate or competing resolution certificates cannot exist.

### Rule 6: Immutable Status Audit Trail

- Every single status change—from initial submission to resolution or closure—automatically generates a new record in `complaint_history` .
- The record permanently preserves the `old_status` , `new_status` , administrative remarks, authoring officer, and update timestamp.
- History records are append-only and cannot be altered or deleted, ensuring full auditability.

### Rule 7: Foreign Key Enforcement & DB Browser Compatibility

- Every SQLite database connection must explicitly execute: `PRAGMA foreign_keys = ON;` This guarantees that no orphaned complaints can be created for non-existent citizens, categories, locations, or departments.
- The database file `municipal_complaints.db` maintains complete binary compatibility with standard SQLite tooling, specifically **DB Browser for SQLite**, allowing live table inspection, visual schema diagramming, and direct SQL querying during hackathon judging.

---